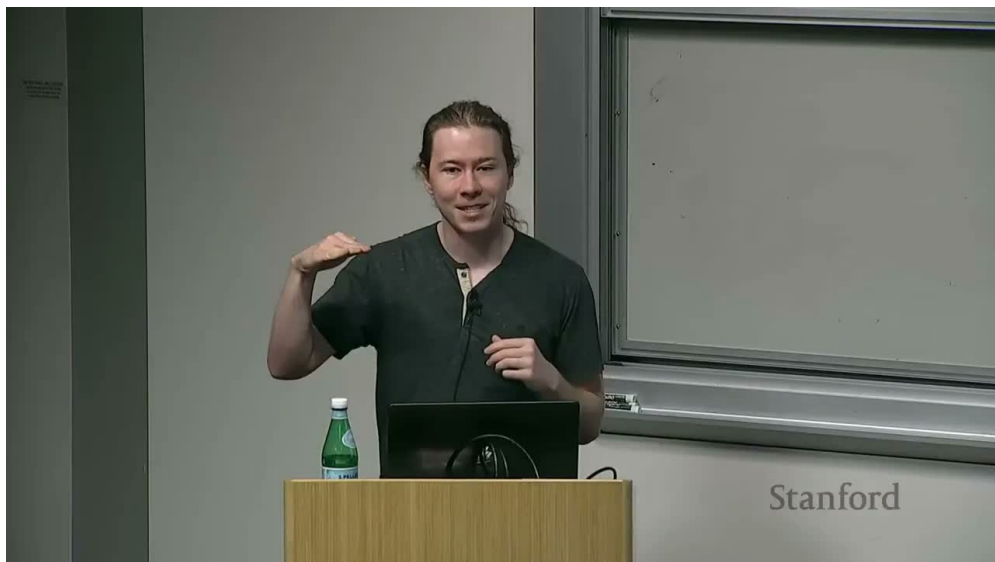


课程笔记

CS231n Lecture 11: Large Scale Distributed Training

Codex Harness

2026-04-15



目录

1	为什么大模型训练必然走向分布式	2
1.1	本章小结	2
2	数据并行: 最自然但不够用的第一步	2
2.1	本章小结	3
3	FSDP 与显存优化: 把状态切开保存	3
3.1	本章小结	4
4	Activation Checkpointing, HFU 与 MFU	4
4.1	本章小结	5
5	Context, Pipeline, Tensor: 把模型沿不同维度切开	5
5.1	本章小结	6
6	总结与延伸	6

1 为什么大模型训练必然走向分布式

这讲的出发点非常实际：以 Llama3-405B 这类超大模型为例，单张 GPU 无论在显存、吞吐还是通信带宽上都已经无法独立承担训练任务。课程因此把一个 cluster 重新解释成“一台由很多 GPU 和高速互联组成的大计算机”。

这个视角很重要，因为它改变了训练问题的表述方式。我们不再问“怎么在一张卡上跑起来”，而是问：

- 数据应当切在哪里？
- 模型参数应当切在哪里？
- 序列与时间应当切在哪里？
- 通信与计算应当怎样重叠？

1.1 本章小结

大规模训练的本质，不是简单地“多加几张卡”，而是把整个训练系统重新设计成一套分布式程序。

2 数据并行：最自然但不够用的第一步

数据并行（Data Parallelism）是最自然的起点：每张 GPU 持有一份完整模型副本，但各自处理不同 mini-batch 切片。

如果总 batch 大小为 B ，每张卡上的局部梯度记作 $g^{(n)}$ ，那么聚合后的全局梯度可以写成：

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B \ell(x_i; \theta), \quad g = \frac{1}{N} \sum_{n=1}^N g^{(n)}$$

这里的关键操作就是 **all-reduce**：每张卡先独立完成 forward/backward，再把梯度求和或求均值，最后同步更新参数。

Data Parallelism

Recall: Loss is usually averaged over a minibatch of N samples

图 1: 数据并行把样本分到多卡，但每张卡仍保留完整模型副本。

```

1 for local_batch in loader_shard:
2     loss = model(local_batch).loss()
3     loss.backward()
4     all_reduce_gradients(model.parameters())
5     optimizer.step()
6     optimizer.zero_grad()

```

Listing 1: 数据并行训练 step 的最小伪代码

它的优点是概念简单、易于扩展；但它也很快触碰上限。原因很直接：**模型参数、梯度、优化器状态都被完整复制在每张卡上**。当模型足够大时，仅复制开销就会压垮显存。

2.1 本章小结

数据并行解决的是算力扩展问题，但没有解决模型状态复制问题。

3 FSDP 与显存优化：把状态切开保存

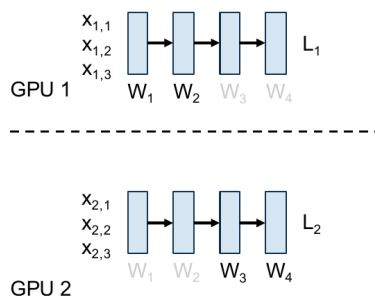
当数据并行不再够用，下一步自然是对模型状态做分片。课程在 slides 中出现了“FSPD”的写法，但其实际内容对应的是 **Fully Sharded Data Parallelism, FSDP**。

FSDP 的核心思想是：不要让每张卡都完整保存参数、梯度和优化器状态，而是把这些状态切开存。这样每张卡只负责自己那一片，在需要计算某层时再做 gather，在计算结束后再释放或重新切分。

Fully Sharded Data Parallelism (FSDP)

Split model weights across GPUs

Each weight W_i is owned by one GPU, which also holds its grads and optim states



Rajbhandari et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models", arXiv 2019

Stanford CS231n 10th Anniversary

Lecture 11 - 46

May 6, 2025

图 2: FSDP 把参数、梯度和优化器状态切分到不同设备，从复制问题转化为通信与调度问题。

这类方法带来的收益是显著的：

- 更大的模型可以塞进同样数量的 GPU。
- 可用 batch size 往往也随之提升。
- 为后续与 pipeline/tensor/context parallel 组合打下基础。

课程还提醒，很多看似次要的细节也会影响显存：例如最后一层激活是否保留、某些张量生命周期是否拉得过长、是否采用 hybrid sharding 等。分布式训练并不是“选一种 parallelism 就结束”，而是大量低层工程细节的总和。

3.1 本章小结

FSDP 的关键不是“更高级”，而是它首次真正解决了状态复制的根本问题。

4 Activation Checkpointing, HFU 与 MFU

即使做了 sharding，激活 (activations) 本身仍可能成为瓶颈。于是课程引出 **activation checkpointing**：训练时不保存所有中间激活，而是在反向传播需要时重新计算一部分 forward。

Activation Checkpointing

Each layer in the network is two functions:

Forward: Compute next-layer activations

$$A_{i+1} = F_i^{\rightarrow}(A_i)$$

Backward: Compute prev-layer gradients

$$G_i = F_i^{\leftarrow}(A_i, G_{i+1})$$

Q: How much compute and memory does this take? Assume each $F_i^{\rightarrow}$ and $F_i^{\leftarrow}$ is $O(1)$ compute and memory.

Compute: 3
Current Memory: 3
Peak Memory: 3

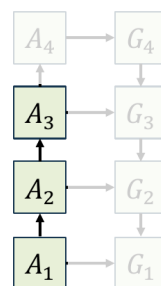


图 3: Activation checkpointing 用额外计算换显存，是大模型训练中的常规工程手段。

这是一种很典型的 tradeoff:

- 少存激活，显存压力下降。
- 反向阶段需要重算，计算量上升。

课程随后区分了两个吞吐指标：

$$\text{HFU} = \frac{t_{\text{theoretical matmul}}}{t_{\text{actual}}}, \quad \text{MFU} = \frac{t_{\text{theoretical model}}}{t_{\text{actual}}}$$

直观上，HFU 更接近“硬件上的矩阵乘法利用率”，而 MFU 更接近“整个模型训练系统实际有多高效”。课程之所以强调 MFU，是因为 checkpointing、helper kernels、通信等待等开销，往往不会被 HFU 充分反映。

为什么不能只看 HFU

某个系统可能 matmul 很忙，看起来 HFU 不差，但如果通信阻塞、重算太多或 pipeline 气泡很大，整体训练速度依然不理想。MFU 更接近端到端系统表现。

4.1 本章小结

checkpointing 解决的是“显存怎么省”，HFU/MFU 回答的是“训练有没有真的跑快”。两者共同决定大规模训练的工程质量。

5 Context, Pipeline, Tensor: 把模型沿不同维度切开

当模型、序列和层数都继续增长时，只靠数据并行和 sharding 仍然不够。课程于是把并行策略扩展到了更多维度。

Context Parallelism 主要针对长上下文 Transformer。它把序列切开，让不同 GPU 负责不同上下文片段，从而缓解长序列注意力的显存压力。

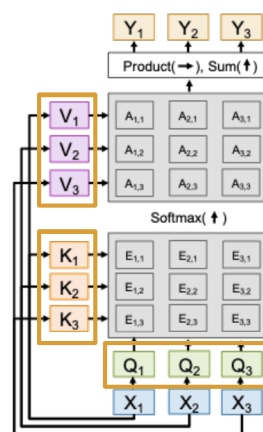
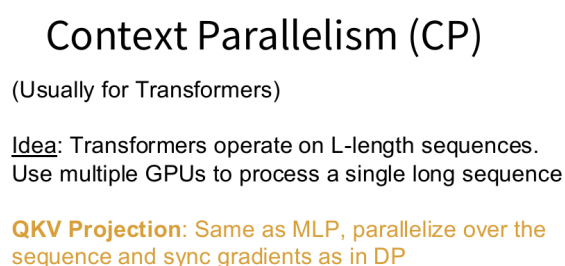


图 4: Context parallel 与 pipeline parallel 表明，模型不仅能沿数据维切，也能沿序列维和层维切。

Pipeline Parallelism 则把模型不同层分配到不同 GPU，上游卡处理完 microbatch 后立刻把中间激活送给下游卡。课程特别强调了 **pipeline bubble**：如果 microbatch 不足或调度不佳，某些设备会空转，最大 MFU 会被流水线深度限制。

Tensor Parallelism 更进一步，它不切样本也不切层，而是直接切单层内部的大矩阵乘法。以 Transformer 为例，projection 和 MLP 中的大矩阵往往就是最适合切分的对象。

Tensor Parallelism (TP)

Idea: Split the weights of each linear layer across GPUs, use block matrix multiply

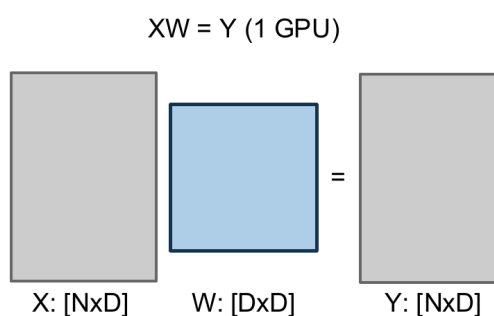


图 5: Tensor parallel 直接把单层内部的大矩阵运算拆到多卡，是扩展超大宽度模型的重要方式。

课程最后用 **ND Parallelism** 收束全讲：真正的大模型训练几乎总是多种并行策略的组合，而不是只选其中一种。数据维、参数维、层维、序列维和算子维都可能同时参与切分。

5.1 本章小结

并行不是单选题，而是多维组合题。真正的系统设计能力，体现在能否把这些维度拼成一个高效而稳定的整体。

6 总结与延伸

这讲的中心思想可以浓缩为一句话：**大规模训练，本质上是把训练程序沿多个维度切开，并用通信把它重新粘起来。**

从数据并行开始，我们看到完整模型复制的上限；从 FSDP 和 checkpointing，我们看到显存如何通过切分和重算换回来；从 context / pipeline / tensor parallel，我们看到模型、序列和层本身都可以成为切分轴。

对后续课程与实践而言，最重要的不是记住每个缩写，而是建立一个系统观：

- 先找出真正的瓶颈是显存、通信还是吞吐。
- 再决定应沿数据、状态、层、序列或算子哪个维度切分。
- 最后用 MFU 这类更接近端到端效率的指标衡量整体方案。